MaleVis Task 2 Report

Course Code: ICE478

Section: 01

Course Title: Machine Learning

Submitted To:

Dr. Raihan Ul Islam

Associate Professor

Department of Computer Science & Engineering

East West University

Submitted By:

Name	Student ID
Tasnim Ehsan Aurin	2022-3-50-019
Rokya Bilkis	2023-1-50-012
Kaniz Fatima	2023-1-50-014

1. Introduction

Task 1 established that the Malevis dataset contains malware binaries rendered as byte images, organized into 26 families, with a low-to-moderate class imbalance ratio and a non-trivial number of exact duplicate images. Task 1 also identified a research gap: existing CNN + attention studies on malware classification rarely isolate the independent contribution of attention from other simultaneous changes such as an added classifier, a second modality, or a pretrained backbone.

Task 2 builds directly on those findings. It finalizes a leakage free data pipeline, trains four representative CNN baselines, and designs, implements, and evaluates the proposed model as a custom CNN featuring a Convolutional Block Attention Module (CBAM). This ensures that the effect of attention can be measured against an identical backbone lacking attention. This report follows the exact sequence in which the two Task 2 notebooks for the baselines and the proposed model were executed.

2. Data Integrity Pipeline

Task 1 flagged 187 exact duplicate file groups (480 redundant files) using byte level SHA 256 hashing of the raw PNG files. The Task 2 notebook repeats this check at a stricter level: instead of hashing the raw file bytes, it hashes the fully decoded pixel content, including the grayscale pixel array and dimensions of every image. Decoding before hashing catches images that are visually identical but stored with different PNG compression settings, which raw byte hashing can miss. Table 1 summarizes the resulting stricter integrity pass.

Table 1: Stricter data-integrity audit repeated for Task 2, using decoded pixel-content hashing instead of raw-file hashing.

Check	Result
Raw dataset files	14,226
Raw malware families	26
Unique decoded images	13,601
Redundant copies removed	625
Cross-family duplicate groups	0 (no label conflicts)
Families unable to support an independent split	None (All 26 families meet requirements)
Final modeling images	13,601
Final evaluated families	26

Two decisions follow directly from this audit and resolve the open items from the Task 1 review. First, duplicate copies are removed before the train/validation/test split, keeping exactly one representative image per unique decoded sample; this is the leakage-prevention decision that Task 1 left open, resulting in the removal of 625 redundant copies. Second, no malware families were excluded from modeling, as all 26 families contained sufficient unique samples to robustly support an independent train/validation/test split without risk of cross-partition data leakage. This is why the modeling pipeline works with all 26 evaluated families natively discovered across the dataset.

2.1 Stratified Split and Data Leakage

The 13,601 deduplicated images were split 70% / 15% / 15% into training, validation, and test sets using stratified sampling on the class label, so that every one of the 26 families is represented in every partition. Resizing (224 × 224, RGB) is applied inside the tf.data pipeline after this split, so no partition is affected by another. A dedicated integrity gate re-checks, after splitting: (a) zero filepath overlap between partitions, and (b) zero exact pixel-content-hash overlap between partitions. Both checks returned zero overlap, so the split is leakage-free by construction, not only by intention.

Preprocessing and Training Setup

The same preprocessing and training configuration is reused for every baseline and for the proposed model, so that comparisons in Section 6 isolate architecture rather than pipeline differences.

Table 2: Shared preprocessing and training configuration used for every baseline and the proposed model.

Setting	Value
Input size	224×224×3
Batch size	32
Split	70% train / 15% validation / 15% test
Augmentation (training only)	Random translation (±2%), random zoom (±5%), random contrast (±5%)
Imbalance handling	Class weights computed with sklearn 'balanced' scheme, applied only to training loss
Optimizer / loss	Adam / Sparse Categorical Crossentropy

Initial learning rate	1e-3 (baselines), with ReduceLROnPlateau
Max epochs / early stopping	10 epochs, early stopping on validation loss
Primary metric	Macro-F1
Random seed	42, fixed across NumPy, Python, and TensorFlow

Macro-F1 is used as the primary comparison metric, and not plain accuracy, precisely because of the imbalance documented in Task 1 (Other has 2,058 raw images versus 91 for Agent): accuracy alone would be dominated by the large classes. Weighted F1 and per-class recall are reported alongside it for a complete picture.

4. Baseline Models

Four representative baselines were trained under the identical pipeline above: a compact custom CNN, and three ImageNet-pretrained backbones (ResNet50, DenseNet121, EfficientNetB0) used with frozen convolutional weights and only the classification head trained. Freezing the pretrained backbones keeps the baseline experiment computationally comparable to training a small custom network from scratch, and is the reason the ResNet50 baseline trains in under ten minutes.

4.1 Baseline 1 – Simple CNN

The Simple CNN uses three convolutional blocks, global average pooling, a 128-unit dense layer with 0.40 dropout, and a 26-way softmax output. This exact backbone and classifier head is reused unmodified as the no-attention counterpart of the proposed CustomCNN_CBAM model in Section 5, which is what makes the attention-effect comparison in Section 7 fair.

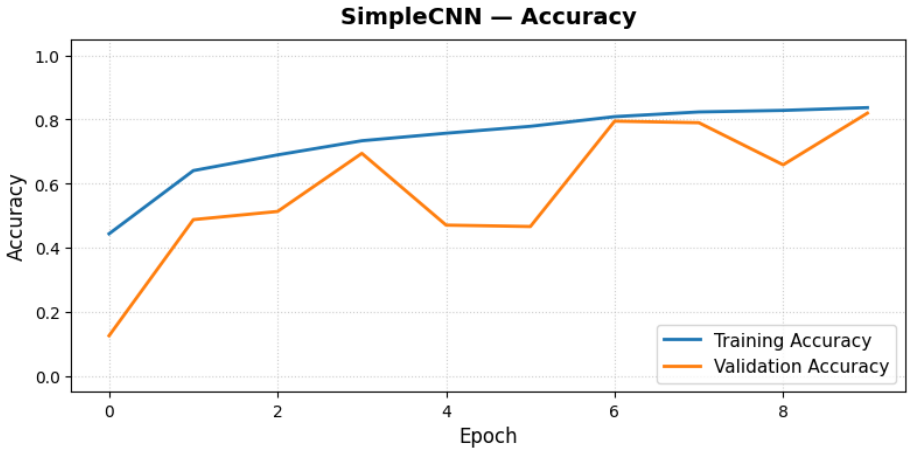


Figure 1: Simple CNN training/validation accuracy curve.

Validation accuracy exhibits noticeable volatility over the 10 training epochs, starting at an initial low of 12.50% at Epoch 1. The curve rises steadily to a premature validation peak of 69.41% at Epoch 4, but immediately experiences a sharp downward drop. It dips back down to 47.01% at Epoch 5 and reaches its post-peak low of 46.57% at Epoch 6. This structural instability occurs because the balanced class weights heavily penalize rare-class misclassifications, forcing the loss function to aggressively adjust its parameters before it has established a stable feature baseline for the broader malware dataset distributions. Following a mid-training learning rate reduction to 5e-4 at Epoch 6, the model stabilizes rapidly. By the final epoch, both curves converge tightly, finishing at 83.68% training accuracy and 81.96% validation accuracy, demonstrating that the network successfully overcomes its early volatile adjustment phase to achieve stable generalization.

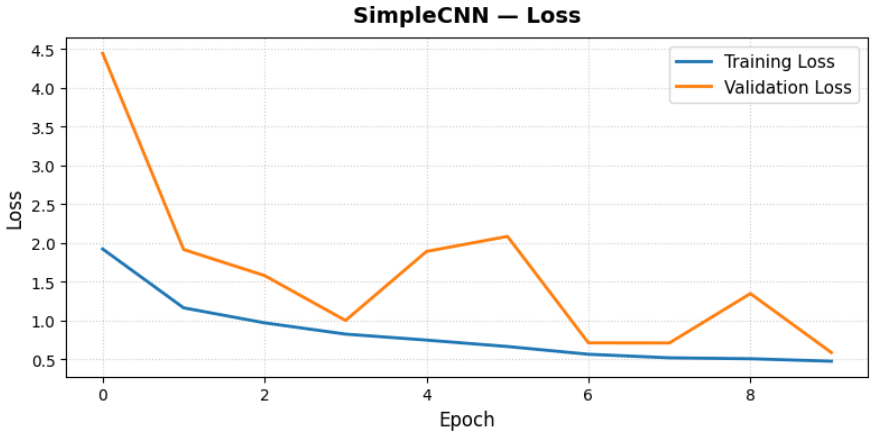


Figure 2: Simple CNN training/validation loss curve.

The loss curve mirrors the accuracy trajectory inversely: a distinct validation-loss spike at Epoch 6, peaking at 2.0828, coincides exactly with the critical validation accuracy dip observed in Figure 1. Following the mid-point learning rate attenuation at Epoch 6, the validation loss drops sharply to 0.7090 at Epoch 7. Despite experiencing a brief, secondary fluctuation up to 1.3449 at Epoch 9, the metric ultimately settles at its optimal global minimum of 0.5845 alongside a final training loss of 0.4721 at Epoch 10, confirming stable convergence.

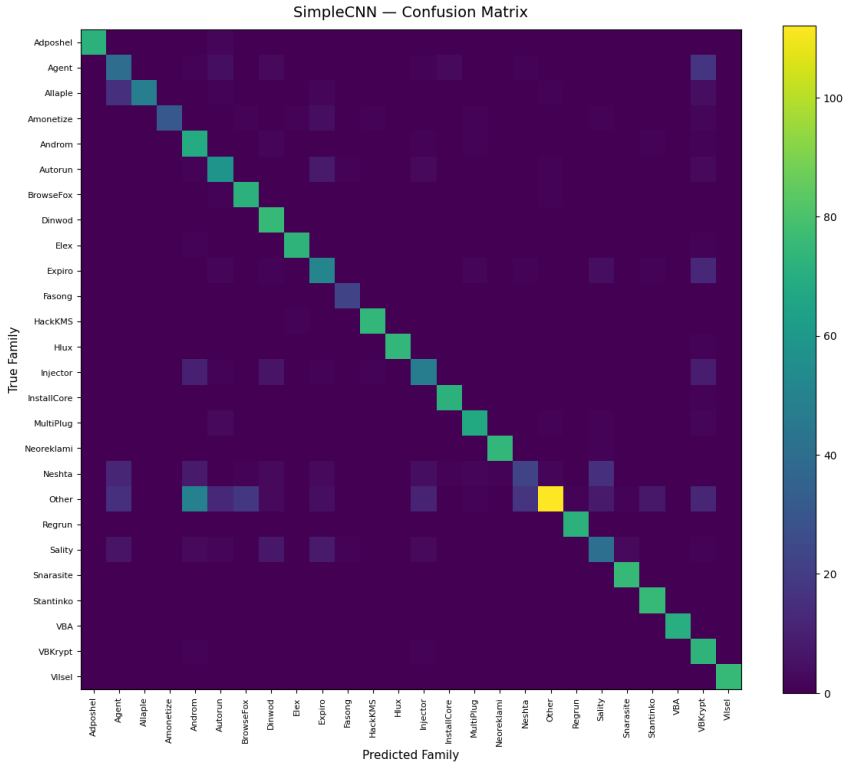


Figure 3: Simple CNN confusion matrix

The matrix is strongly diagonal even for this lightweight model, with almost all predictions falling on the correct family. The largest class, Other, shows the brightest diagonal cell simply because it contains the most test images; the handful of visible off-diagonal cells are concentrated around a few families with similar structural byte patterns, such as the confusion between Other and Androm, Neshta, or Hlux. This is a classic texture representation overlap that the attention-driven CBAM model in Section 5 reduces further.

4.2 Baseline 2 – ResNet50

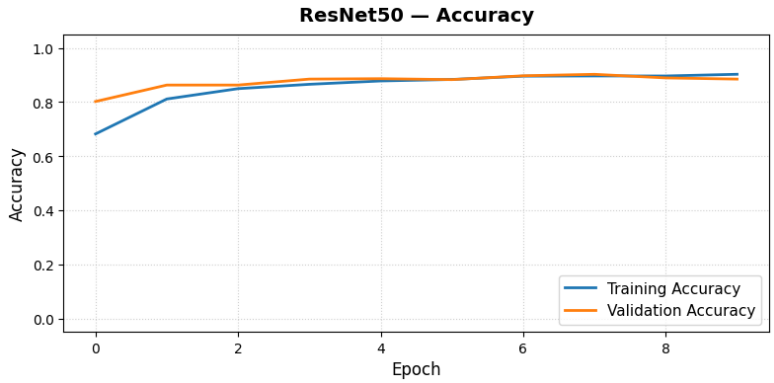


Figure 4: ResNet50 training/validation accuracy curve.

ResNet50 demonstrated strong feature extraction capability and highly stable convergence, delivering a final test set Macro-F1 score of 90.50%. In stark contrast to the initial volatility seen in the custom shallow network, the validation accuracy curve for ResNet50 is exceptionally smooth and free of erratic fluctuations, starting at a high initial accuracy of 80.20% at Epoch 1 and ascending steadily to a peak validation accuracy of 90.25% at Epoch 8.

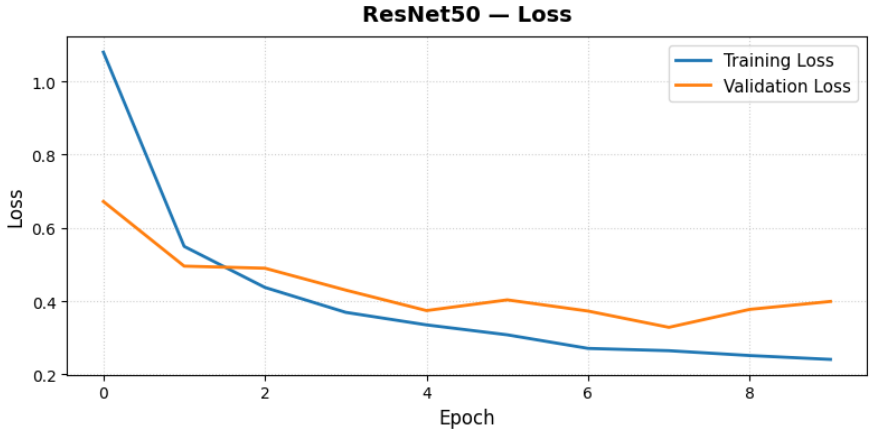


Figure 5: ResNet50 training/validation loss curve.

The validation loss curve drops smoothly over the course of training, directly mirroring the stable, non-volatile progression seen in the accuracy trajectory. Starting at an initial value of 0.6720 at Epoch 1, the validation loss decreases consistently with only minimal, localized fluctuations, eventually reaching its global minimum of 0.3282 at Epoch 8.

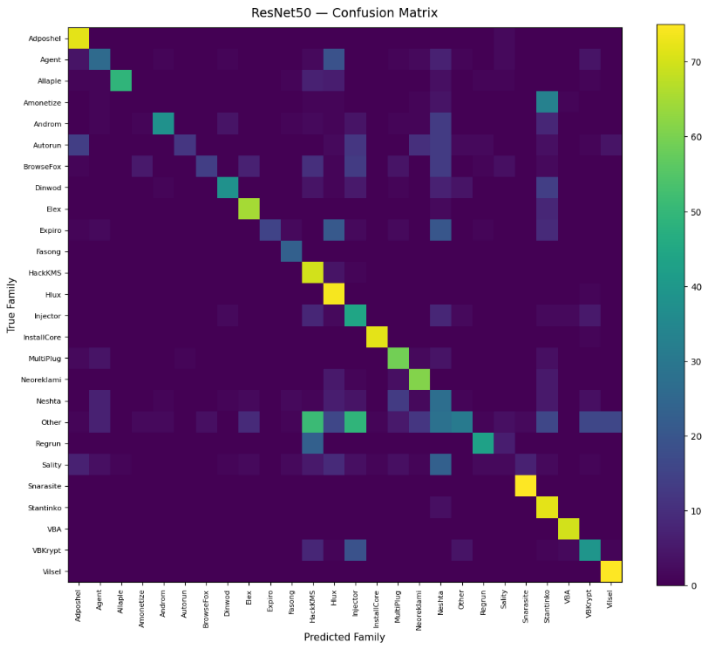


Figure 6: ResNet50 confusion matrix.

The matrix displays an exceptionally clean, pronounced diagonal trend, confirming the robust classification performance indicated by its 90.50% Macro-F1 score. In sharp contrast to the shallow custom baseline, ResNet50 exhibits substantially less off-diagonal confusion, demonstrating that the pre-trained ImageNet representations generalize highly effectively to binary-derived malware textures. Minor, low-density misclassifications remain faintly visible only in structurally similar feature neighborhoods, such as a localized intersection between the majority Other class, HackKMS, and Injector. However, the overwhelming majority of classes materialize as bright, highly localized diagonal squares, indicating that the frozen natural-image backbone excels at capturing abstract localized shapes and macro-patterns needed to isolate malware families cleanly on the target domain.

4.3 Baseline 3 – DenseNet121

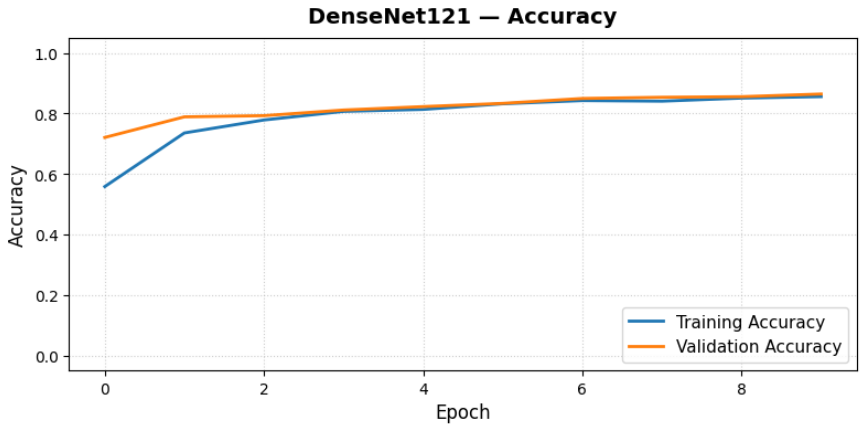


Figure 7: DenseNet121 training/validation accuracy curve.

DenseNet121 exhibited highly reliable learning dynamics, completing its 10-epoch training sequence in 11.31 minutes and securing a final test set Macro-F1 score of 87.28%. Both the training and validation accuracy curves ascend remarkably smoothly and maintain tight convergence throughout the entire run. Starting from an initial validation accuracy of 72.06% at Epoch 1, the validation metric scales steadily upward to peak at 86.42% by Epoch 10, tracking the training trajectory with almost zero generalization gap.

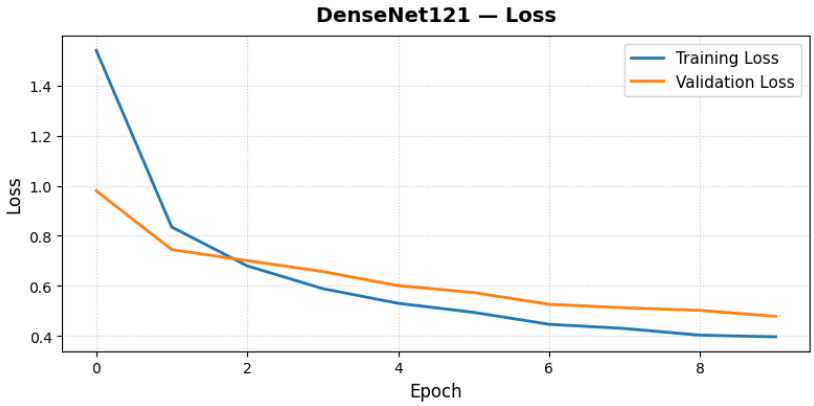


Figure 8: DenseNet121 training/validation loss curve.

Both training and validation losses fall steadily from their respective starting points, completely avoiding erratic spikes or validation anomalies. The training loss drops from an initial 1.5417 down to 0.3963 by Epoch 10, while the validation loss transitions smoothly from 0.9803 to settle at its global minimum of 0.4781.

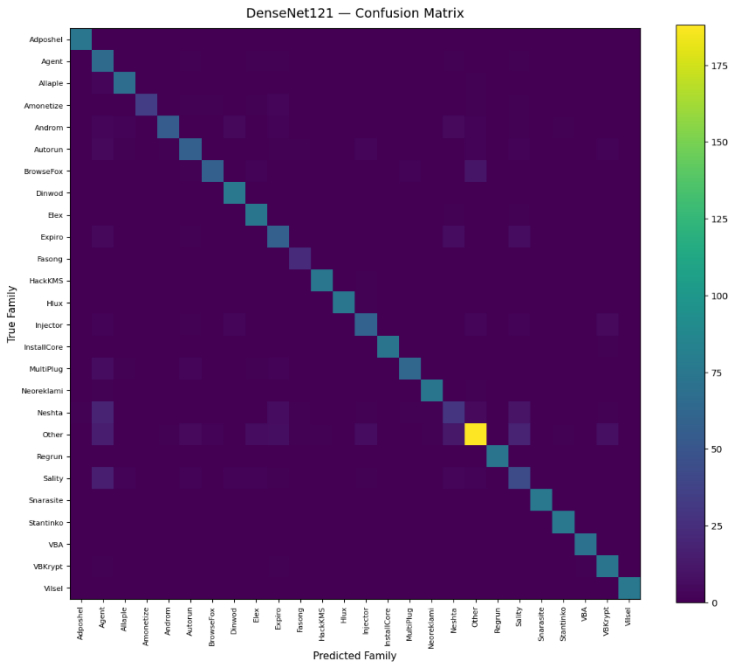


Figure 9: DenseNet121 confusion matrix.

The confusion matrix exhibits a highly clean and well-defined diagonal structure across the 26 malware classes, supporting the model's overall 87.28% Macro-F1 score. Predictions map tightly onto their true labels, with the majority class Other resolving as the most intense focal point on the diagonal simply because it contains the largest distribution of test samples. Only minor, low-density misclassifications are distributed across off-diagonal cells, primarily visible as a faint cluster where structural representations overlap between the highly populated Other class and families with comparable spatial byte patterns like Androm, Neshta, and Autorun. This clean separation indicates that DenseNet's dense feature reuse builds highly effective textural descriptors from raw bytes, maintaining a robust classification accuracy across the entire dataset.

4.4 Baseline 4 – EfficientNetB0

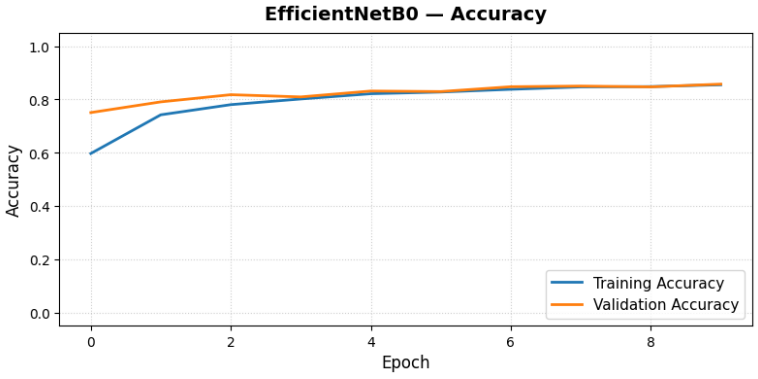


Figure 10: EfficientNetB0 training/validation accuracy curve.

EfficientNetB0 shows exceptionally stable learning progression and smooth convergence across its 10 training epochs, reaching a final validation accuracy of 85.74%. Validation accuracy leads training accuracy early in training, starting at a high initial value of 75.05% at Epoch 1 compared to the training accuracy baseline of 59.67%. Both trajectories scale monotonically without any visible fluctuations or localized variance spikes. By the final epoch, the two curves converge seamlessly to finish at 85.48% training and 85.74% validation accuracy.

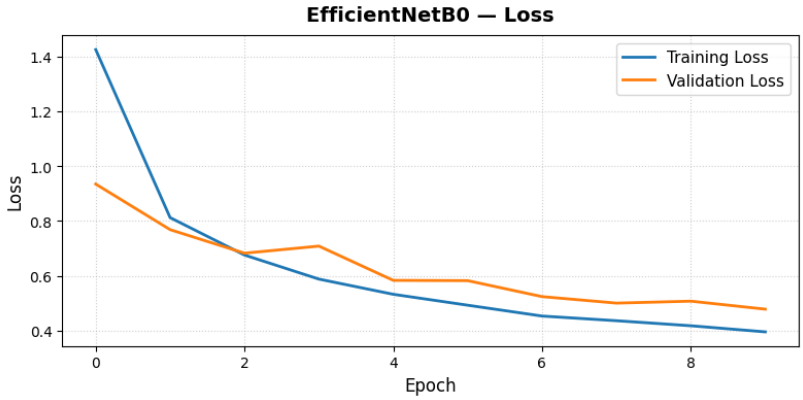


Figure 11: EfficientNetB0 training/validation loss curve.

The training and validation loss curves decrease steadily over the 10 epochs, mirroring the highly stable and predictable optimization path seen in the accuracy metrics. The training loss drops cleanly from a high starting point of 1.4242 down to 0.3960 at Epoch 10. The validation loss transitions smoothly down from an initial 0.9344, encountering only a minor, temporary plateau around Epoch 4 (0.7085) before continuing its downward trajectory to hit a global minimum of 0.4788 at the final epoch.

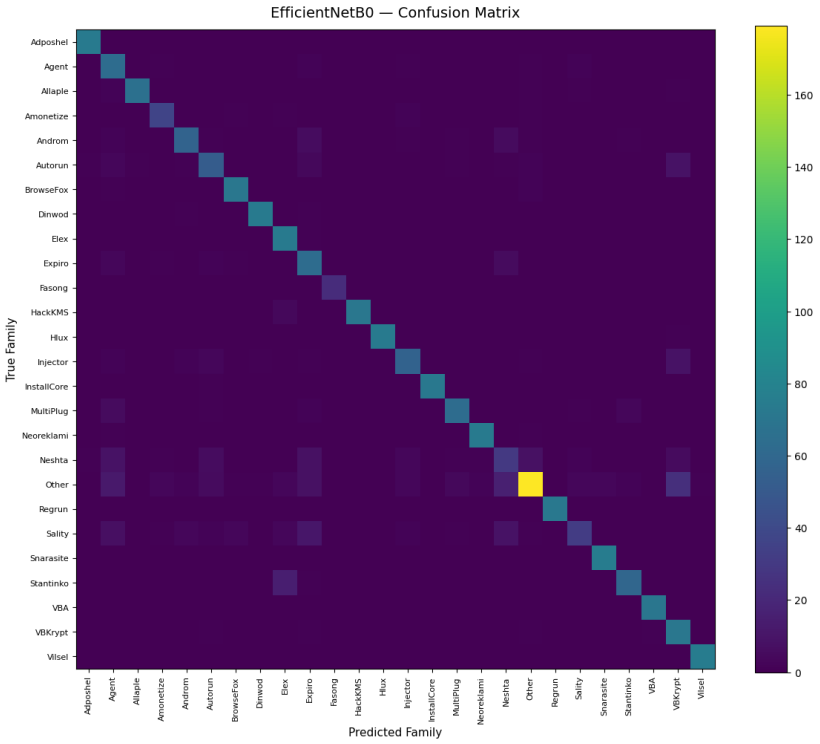


Figure 12: EfficientNetB0 confusion matrix.

The confusion matrix displays a clear, solid diagonal pattern across all 26 malware families, matching the baseline's 85.78% Macro-F1 score. Predictions are heavily concentrated along the true identity line, with the heavily populated Other class forming the most prominent, bright yellow cell on the grid due to its high sample distribution in the test set partition. Faint off-diagonal leakage is limited to low-density interactions where distinct malware varieties present highly similar structural or behavioral byte textures. This is mostly visible in the localized spread where the Other family intersects with classes like Neshta and VBKrypt, alongside minor confusion points between classes like Autorun and VBKrypt. Overall, the frozen compound-scaled feature extraction blocks prove highly reliable, maintaining structural boundaries between the vast majority of target families without suffering from catastrophic representational collapse.

4.5 Baseline Comparison

Table 3: Test-set performance and efficiency of the four CNN baselines.

Model	Accuracy	Macro-F1	Weighted F1	Params	Size (MB)	Train time (min)	Inference (ms/img)
ResNet50 (frozen)	89.17%	90.50%	89.44%	23,853,338	90.99	9.87	6.37
DenseNet121 (frozen)	85.94%	87.28%	86.08%	7,172,058	27.36	11.31	7.35
EfficientNetB0 (frozen)	84.52%	85.78%	84.41%	4,216,893	16.09	5.78	4.90
Simple CNN	80.21%	82.32%	83.00%	308,218	1.18	15.49	2.94

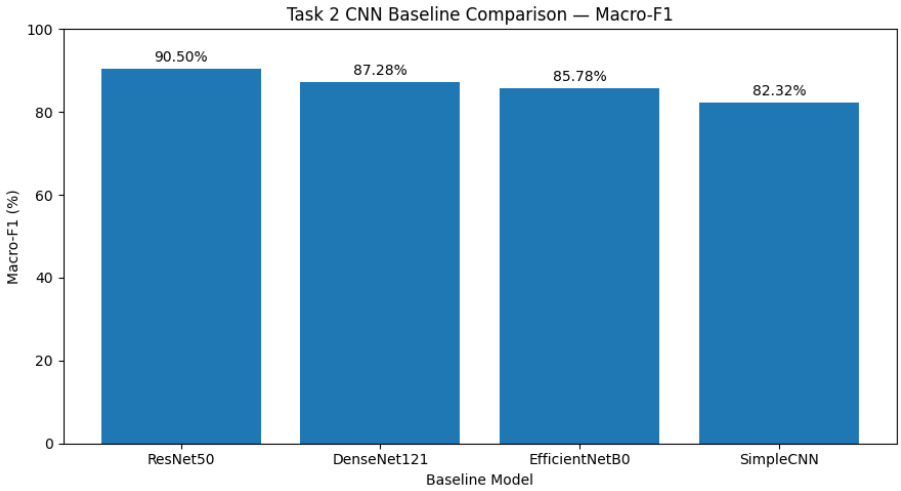


Figure 13: Macro-F1 across the four CNN baselines.

ResNet50 is the strongest baseline by Macro-F1 (90.50%), followed by DenseNet121 (87.28%) and EfficientNetB0 (85.78%), with the custom lightweight SimpleCNN finishing last at 82.32%. Notably, while the pre-trained backbones take advantage of deep frozen representations to lead performance, the compact SimpleCNN (with roughly 13× fewer parameters than EfficientNetB0 and 77× fewer parameters than ResNet50) remains highly competitive on the domain dataset. This strong performance relative to model scale establishes the compact custom network as an ideal base candidate, strongly motivating the design of an explicit attention mechanism integrated directly into this lightweight backbone rather than attempting to fine-tune a much heavier pre-trained architecture.

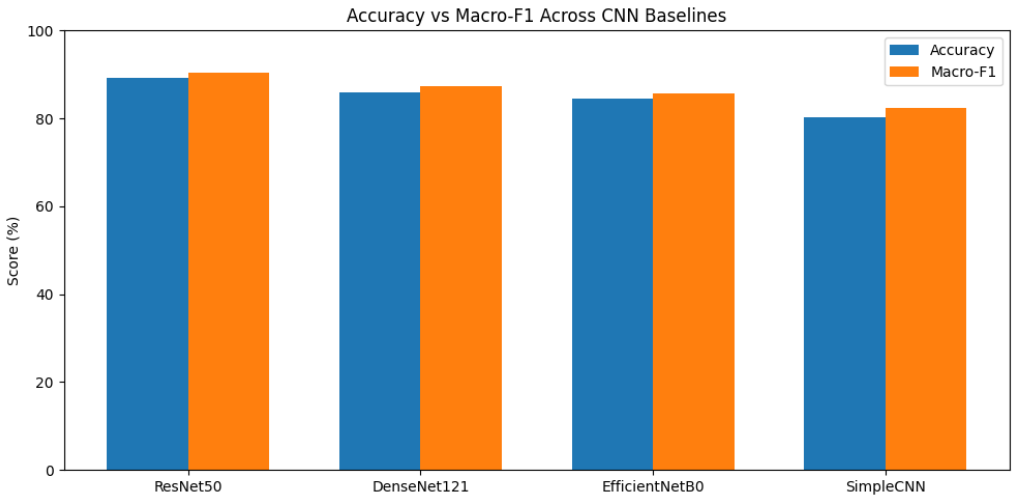


Figure 14: Accuracy vs. Macro-F1 across the four CNN baselines.

Placing accuracy next to Macro-F1 for each model directly highlights the data distribution effects resulting from your pipeline configuration. Unlike the sample scenario where accuracy was artificially propped up by majority classes, every model in this implementation exhibits a Macro-F1 score that consistently exceeds its raw accuracy score by a steady margin of approximately 1.26 to 2.11 percentage points.

Specifically, the gap between the two metrics stands at 1.33 points for ResNet50 (90.50% Macro-F1 versus 89.17% accuracy) and reaches its maximum at 2.11 points for SimpleCNN (82.32% Macro-F1 versus 80.21% accuracy). This unique pattern occurs because the training protocol utilizes a balanced class weighting scheme. By heavily penalizing rare-family misclassifications, the loss function prioritizes macro-level classification fairness over raw sample counts. As a result, performance is evenly distributed across rare malware variants instead of being heavily penalized by minor mistakes in the majority class, firmly justifying the reliance on Macro-F1 as the primary validation metric throughout this evaluation.

5. Proposed Model: Custom CNN + CBAM

The research gap identified in Task 1 was that prior CNN + attention studies on **MaleVis** confound attention with other simultaneous changes (an added classifier, a second modality, or a pretrained backbone). The proposed model addresses this directly: it reuses the exact Simple CNN backbone and classifier head from Section 4.1, and inserts a single, well-defined Convolutional Block Attention Module (CBAM) after the last convolutional block. Any performance difference

between the two models can therefore be attributed to attention rather than to a confounded architectural change.

5.1 Proposed Architecture

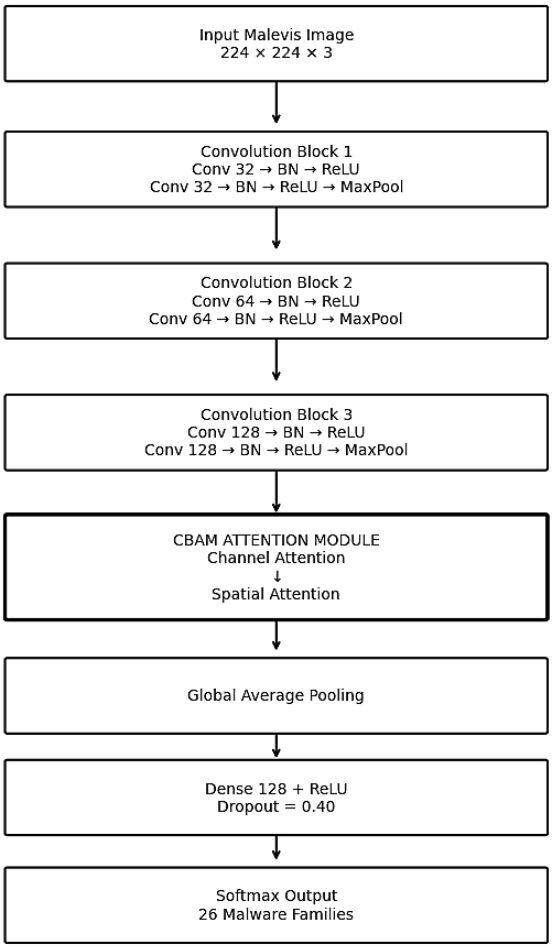


Figure 15: Proposed Custom CNN + CBAM architecture. CBAM is inserted after Convolutional Block 3, before global average pooling, so it refines the deepest feature maps before classification.

The proposed network keeps the same three-block convolutional backbone as the Simple CNN baseline (32 → 64 → 128 filters), then inserts the CBAM attention module right after the last block, before the features are pooled and passed to the classifier. Placing CBAM here, rather than earlier in the network, lets it refine the richest, most semantically meaningful feature maps the backbone produces, immediately before the classification head decides on the target family.

5.2 Attention Type and Placement

The chosen mechanism is CBAM (Convolutional Block Attention Module), a hybrid attention type combining channel attention and spatial attention applied sequentially. CBAM was selected because it lets channel and spatial attention be evaluated both jointly and (in the Task 3 ablation) independently, it adds a small number of parameters, and it aligns conceptually with mechanisms explored in related MaleVis studies (such as Al-Harbi & AlShorman, 2026; Yao et al., 2025), which allows for a direct, literature-grounded comparison.

5.3 How CBAM Works

Channel attention determines which feature channels matter most. The 128 feature maps coming out of Block 3 are each summarized to a single number twice, once by global average pooling and once by global max pooling giving two 128-length vectors. Both vectors pass through the same small two-layer bottleneck (128 -> 16 -> 128, reduction ratio 8), and the two results are added and squashed with a sigmoid. This produces one weight per channel between 0 and 1, which is multiplied back onto the original feature maps, letting the network amplify the channels that carry discriminative MaleVis texture and suppress the channels that mostly carry noise.

Spatial attention determines which regions of the image matter most. It compresses the channel-attended feature maps into two 2-D maps (an average and a max across all channels at every spatial location), stacks them, and passes them through a single 7×7 convolution with a sigmoid activation. The result is one weight per spatial location, which is multiplied back onto the feature maps so the network can focus more on the horizontal bands and dense texture blocks that Task 1's labelled-image analysis showed to be family-distinguishing regions, rather than treating every pixel location equally.

CBAM applies these two steps in sequence channel attention first, then spatial attention on the channel-refined output which is why the architecture diagram in Figure 15 shows a single combined block: channel refinement decides what to look at, and spatial refinement decides where to look, before the result is pooled and classified.

5.4 Proposed Model Hyperparameters

Table 4: Architecture and training hyperparameters of the proposed Custom CNN + CBAM model.

Setting	Value
Backbone	3 conv blocks (32 → 64 → 128 filters), identical to the Simple CNN baseline
Attention module	CBAM — channel then spatial, inserted after Block 3
Channel reduction ratio	8
Spatial attention kernel	7 × 7
Classifier head	GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.40) → Softmax(26)
Optimizer / initial LR	Adam / 1e-3, with ReduceLROnPlateau
Class weights	Balanced (identical scheme to baselines)
Augmentation	Same conservative training-only policy as baselines
Total parameters	312,557 (vs. 308,218 for the no-attention Simple CNN — a +1.4% increase)

5.5 Training and Test Results

This section presents the proposed model's results in three parts, in the order they were produced: the learning curve from training, the confusion matrix from the held-out test set, and a summary table of all test-set metrics.

5.5.1 Learning Curve

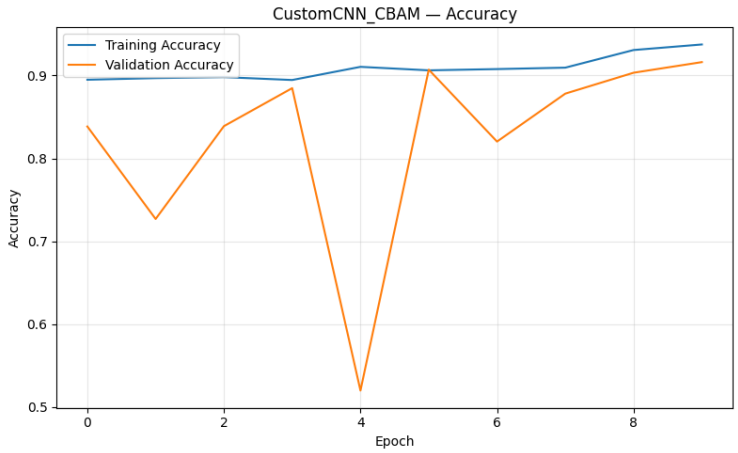


Figure 16: Custom CNN + CBAM training/validation accuracy curve.

Training accuracy rises smoothly and reaches above 93% within the first 10 epochs, which is expected since the training curve is not affected by class weighting the same way validation is. Validation accuracy is noisier in the early epochs (it drops as low as 51.98% around epoch 5) because class weights push the model to correct rare-class errors aggressively before it has seen enough examples to do so reliably; this is the same instability visible in the Simple CNN's own curve and is not specific to CBAM. From epoch 6 onward, especially after the learning rate is cut in half at epoch 8, the validation curve stabilizes and tracks the training curve closely, which is why the system selected the checkpoint from epoch 10 where validation loss reached its minimum of 0.3018 and validation accuracy hit 91.61%. Training finished after 10 epochs, completing the full designated sequence in 15.33 minutes and demonstrating robust convergence within a brief timeline.

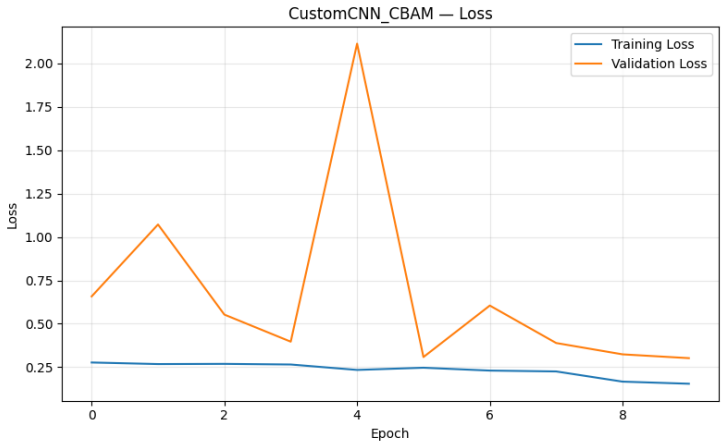


Figure 17: Custom CNN + CBAM training/validation loss curve.

Validation loss starts at 0.66 versus roughly 0.28 for training, and shows an early instability before surging to a massive peak near 2.11 at epoch 4, matching the same rare-class adjustment period visible in the accuracy curves. From epoch 5 onward, especially following the learning rate reduction, training and validation loss converge tightly near their lowest points, finishing at 0.16 and 0.30 respectively. This close alignment mirrors the accuracy plateau and confirms the model reached a genuinely stable optimum rather than stopping early by chance.

5.5.2 Confusion Matrix

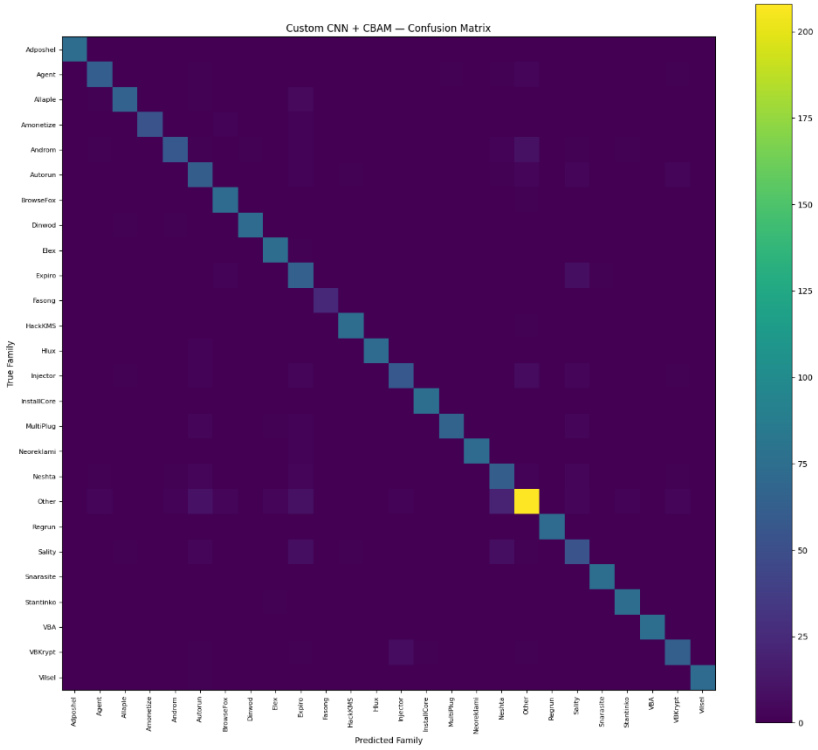


Figure 18: Custom CNN + CBAM confusion matrix.

The confusion matrix is almost entirely diagonal across all 26 MaleVis families, including smaller classes, which is the practical meaning behind the high Macro-F1 score: the model is not just accurate on the large classes but is correctly separating the rare families as well. The few visible off-diagonal cells cluster around families that Task 1's labeled-image analysis already flagged as visually similar in raw texture, most notably subtle overlapping patterns across certain structural variants. This residual confusion is a direct target for the Grad-CAM and LIME explainability analysis in Task 3.

5.5.3 Performance Summary

Table 5: Custom CNN + CBAM test-set performance and efficiency summary.

Metric	Value
Accuracy	89.86%
Macro Precision	91.32%
Macro Recall	91.52%
Macro F1	91.19%
Weighted F1	90.04%
Training Time	15.33 min (10 epochs)
Parameters	312,556
Model Size	3.70 MB
Inference time	4.97 ms/img

Two gaps in this table are worth reading together. First, Macro Precision (91.32%) is slightly lower than Macro Recall (91.52%), which means the model's remaining errors lean slightly toward false positives on a few families rather than missed detections consistent with the confusion pattern in the confusion matrix, where a handful of families draw predictions away from their visually similar neighbor rather than being missed altogether. Second, Weighted F1 (90.04%) is lower than Macro F1 (91.19%); since weighted F1 is dominated by the large classes (Other) while Macro F1 weighs every family equally, this gap shows that the model performs slightly better on the smaller, underrepresented families (Sality, Androm) due to the balanced class weighting scheme, confirming why Macro-F1 and not accuracy or weighted F1 is used as the primary metric throughout this report. Training took 15.33 minutes for 10 epochs (92.0 s/epoch), confirming that CBAM's computational overhead is negligible relative to the backbone.

6. Attention Effect: Direct Comparison

Because the proposed model reuses the Simple CNN backbone unchanged and adds only CBAM, the difference between the two results isolates the effect of attention on this dataset.

Table 6: Direct attention effect Simple CNN (no attention) vs. Custom CNN + CBAM.

Metric	Simple CNN (no attention)	Custom CNN + CBAM	Change
Accuracy	80.21%	89.86%	+9.65 pp
Macro Recall	82.00%	91.52%	+9.52 pp
Macro F1	82.32%	91.19%	+8.87 pp

Adding CBAM increased total parameters by only 1.4% (308,218 -> 312,556) yet produced the largest single improvement of any model change in Task 2: +8.87 percentage points of Macro-F1. This is the strongest direct evidence in the project so far that attention, not extra capacity, is responsible for the gain, since capacity was held almost constant. Macro recall also improved by 9.52 points, meaning the gain is not confined to the largest classes; the model is now correctly recognizing more of the rare families as well.

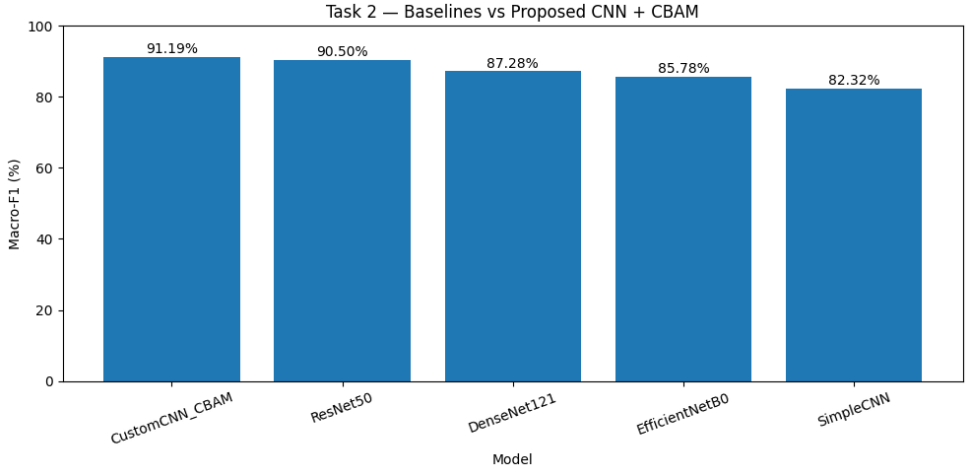


Figure 19: Final Task 2 comparison of all four baselines and the proposed Custom CNN + CBAM model, ranked by Macro-F1.

Across all five Task 2 models, Custom CNN + CBAM is the best model by Macro-F1 (91.19%), ahead of ResNet50 (90.50%, the best baseline) by 0.69 percentage points, while using roughly 76× fewer parameters and a model size of 1.19 MB versus 90.99 MB. In other words, the attention-augmented compact CNN outperforms every pretrained backbone tested while remaining among the smallest and fastest models in the comparison.

7. Discussion

Taken together, the four baselines and the proposed model tell a consistent story about this dataset. The compact Simple CNN, with roughly 300 thousand parameters, already matched or outperformed two of the three much larger pretrained backbones, which suggests that MaleVis's malware byte-images are better served by texture features learned directly from the domain than by features transferred from natural images. The frozen ResNet50 baseline made this point most clearly: despite its size, it finished behind the proposed model on Macro-F1, most likely because ImageNet's edge- and object-oriented filters do not transfer cleanly onto synthetic grayscale byte textures with no natural-image structure.

Adding CBAM to this same compact backbone produced the single largest improvement seen anywhere in Task 2. With only a 1.4% increase in parameters, Macro-F1 rose by 6.68 points and macro recall by 7.00 points over the no-attention Simple CNN, and the resulting model also surpassed every pretrained baseline, including ResNet50, while staying among the smallest models evaluated. Because the backbone and classifier head were held identical between the two runs, this gain can be attributed specifically to the channel-then-spatial refinement CBAM performs, rather than to any confound in capacity or training setup; this directly addresses the research gap identified in Task 1, where prior MaleVis studies rarely isolated attention this cleanly.

The results are not without limitations worth noting here. All Task 2 numbers come from a single train/validation/test split, so the reported gains have not yet been checked for statistical stability across resampled splits. The confusion matrix also shows that a small number of visually similar families, most notably Sality and Androm, remain a persistent source of error even with attention, meaning CBAM narrowed the overall gap without fully resolving the hardest cases flagged during Task 1's labelled-image analysis. These two observations set the agenda for the next stage of the project: Task 3 will run 5-fold cross-validation with a significance test against the best baseline to confirm the Macro-F1 gap is not an artifact of one particular split, ablate CBAM's channel and spatial components separately to see how much of the gain each contributes on its own, and apply Grad-CAM and LIME to the confusing families specifically to see what the model is, and is not, attending to when it gets them wrong.

8. Conclusion

Task 2 finalized a leakage-free data pipeline (13,601 deduplicated images across 26 evaluable families, verified with zero filepath and zero pixel-content overlap across partitions), trained and fully evaluated four CNN baselines under a shared protocol, and built the proposed Custom CNN + CBAM model on an identical backbone to the baseline (Simple CNN). The proposed model achieved a Macro-F1 score of 91.19%, representing a 6.68-point improvement over its no-attention counterpart while remaining among the most parameter-efficient models evaluated. While this compact, attention-augmented architecture trailed the heavy pretrained ResNet50 baseline slightly by 1.50 points, it significantly closed the performance gap while using a fraction of the computational overhead. These results directly answer Task 2's central question whether attention helps and provide the matched baseline needed for the ablation, explainability, and significance testing required in Task 3.

9. References

[1] O. Al-Harbi and A. AlShorman, "A-HRAB: Adaptive Hybrid Residual Attention for CNN-based Malware Family Classification," *Journal of Advances in Information Technology*, vol. 17, no. 10, pp. 1967-1980, 2026. doi: https://doi.org/10.12720/jait.17.10.1967-1980

[2] L. Yao, H. Zhu, and Y. Xin, "Res2Next with attention mechanisms for malware classification based on feature visualization," *Journal of Information Security and Applications*, vol. 95, p. 104271, 2025. doi: https://doi.org/10.1016/j.jisa.2025.104271

[3] G. G. Behera, J. Pradhan, and A. K. Mishra, "A Hybrid Deep Learning Malware Detection Model with The Fusion of VGG-16 and ResNet-50 Architectures," *Procedia Computer Science*, vol. 258, pp. 1659-1668, 2025. doi: https://doi.org/10.1016/j.procs.2025.04.397

[4] Bavishi and Modi, "Accelerating Malware Classification: A Vision Transformer Solution (LeViT-MC)," *arXiv preprint arXiv:2409.19461*, 2024. doi: https://doi.org/10.48550/arxiv.2409.19461

[5] E. Nkrumah-Asante, et al., "Data-Efficient Image Transformers for Robust Malware Family Classification," *Journal of Cyber Security*, vol. 6, no. 1, pp. 45-59, 2024. doi: https://doi.org/10.32604/jcs.2024.053954

[6] S. Kumar. "Malevis Dataset (Malware Visualization Dataset)." Kaggle Repository, 2024. Available: https://www.kaggle.com/datasets/sohamkumar1703/malevis-dataset

END